

Eliminating Exponential Key Growth in PRG-Based Distributed Point Functions

Marc Damie^{1,2}^{*}, Florian Hahn¹, Andreas Peter³, and Jan Ramon²

¹ University of Twente, The Netherlands

² Inria, France

³ Carl von Ossietzky Universität Oldenburg, Germany

Abstract. Distributed Point Functions (DPFs) enable sharing secret point functions across multiple parties, supporting privacy-preserving technologies such as Private Information Retrieval, and anonymous communications. While 2-party PRG-based schemes with logarithmic key sizes have been known for a decade, extending these solutions to multi-party settings has proven challenging. In particular, PRG-based multi-party DPFs have historically struggled with practicality due to key sizes growing exponentially with the number of parties and the field size. Our work addresses this efficiency bottleneck by optimizing the PRG-based multi-party DPF scheme of Boyle et al. (EUROCRYPT’15). By leveraging the honest-majority assumption, we eliminate the exponential factor present in this scheme. Our construction is the first PRG-based multi-party DPF scheme with practical key sizes, and provides key up to $3\times$ smaller than the best known multi-party DPF. This work demonstrates that with careful optimization, PRG-based multi-party DPFs can achieve practical performances, and even obtain top performances.

Keywords: Distributed Point Function · Function Secret Sharing · Private Information Retrieval · Multi-Party Computations.

1 Introduction

Function Secret Sharing [2] is a cryptographic primitive enabling to share secret functions. In these protocols, a key dealer knowing a secret function f distributes p keys to different shareholder. Each shareholder can use its key to obtain a share of $f(x)$, without any communication between the shareholders.

Among all function families, schemes supporting point functions (i.e., $f(x) = \beta$ if $x = \alpha$, 0 otherwise) attracted a lot of attention thanks to their numerous applications notably in Private Information Retrieval (PIR) [7], in anonymous communications [6], in digital currencies [9], and machine learning [4]. These schemes are called “Distributed Point Functions” (DPF) [2,7].

To support these applications, there is a significant research incentive aiming to improve existing schemes, notably their key size. DPF efficiency is commonly evaluated based on the influence of the function domain size (N) on the key size.

^{*} Corresponding author: m.f.d.damie@utwente.nl

For two- and three-party DPF, schemes based on PseudoRandom Generators (PRGs) provide logarithmic key sizes [2,9]. However, there is still a lot of active research to obtain similar key sizes for any arbitrary number of parties.

In multi-party DPF, three main approaches have emerged. First, elliptic-curve-based schemes [6,8] offer practical $O(\sqrt{N})$ key sizes, but they require a non-linear share decoding. This non-linearity makes them incompatible with several key applications such as PIR. Second, Boyle et al. [2] presented a dishonest-majority scheme with $O(\sqrt{N})$ key size, and Bunn et al. [5] an honest-majority scheme with $O(\sqrt[4]{N})$ key size. Unfortunately, this asymptotic cost hides an exponential factors q^p (for output shares in $\mathbb{F}_q$ and p parties). This factor makes these schemes impractical for any modulus $q > 210$ (as detailed in Section 4). This problem was identified in existing works [1,8], but has not been solved *yet*.

Finally, Bunn et al. [5] proposed a third approach based on honest-majority to build an information-theoretic (IT) scheme with $O(\sqrt{N})$ key size *and no exponential factor*. This scheme is the only multi-party scheme with practical key sizes supporting all applications. Even though this scheme is practical, solving the efficiency issues of the other schemes could lead to even better performances. As PRGs have lead to logarithmic key sizes in two and three-party schemes, optimizing multi-party PRG-based schemes could be promising.

Our Contributions Our paper optimizes the PRG-based DPF proposed by [2]. Our optimized scheme avoids the exponential factors present in [2] using the honest-majority assumption; obtaining a key size of $O(\sqrt{N \cdot \binom{p}{m+1}} \log q)$ instead of $O(\sqrt{N} q^{\frac{p-1}{2}} \log q)$. Our benchmark shows that our scheme is the first multi-party PRG-based scheme with practical key sizes. We even provide keys up to $3\times$ smaller than the best performing DPF (i.e., the IT DPF by [5]).

Notations Let p be the number of parties/shareholders, m be the number of corrupted parties. Like most FSS works [2,3,5,7,8], we focus on semi-honest adversaries: follow the protocol and infer *passively* secret information.

Let $\mathbb{F}_q$ be a prime field, N be the function domain size, 1^λ a security parameter, and $\llbracket x \rrbracket_i$ be the i -th share of the secret x . Let $\nu = \lceil \sqrt{N} \rceil$ and $C = \binom{p}{m+1}$. Let $G : \{0, 1\}^\lambda \rightarrow \mathbb{G}^\nu$ be a PRG, and $\mathbb{G}$ an Abelian group.

2 Background

Function secret sharing (FSS) [2] enables sharing secret functions between p parties. Each FSS scheme can share function from a specific function family.

A function family $\mathcal{F}$ [1] is a pair $(P_{\mathcal{F}}, E_{\mathcal{F}})$ where $P_{\mathcal{F}} \subseteq \{0, 1\}^*$ is a collection of function descriptions $\hat{f}$, and $E_{\mathcal{F}} : P_{\mathcal{F}} \times \{0, 1\}^* \rightarrow \{0, 1\}^*$ is a polynomial-time algorithm defining the function described by $\hat{f}$; i.e., $f(x) = E_{\mathcal{F}}(\hat{f}, x)$. All functions within a family share the same domain $\mathcal{X}$ and output space $\mathcal{Y}$.

Due to their applications notably in PIR [7] and anonymous communications [6], the most studied function family has been point functions [2,3,5,6,7];

functions f such that $f(x) = \beta$ if $x = \alpha$, and $f(x) = 0$ otherwise. For point functions, the function description is the tuple $(\alpha, \beta) \in P_{\mathcal{F}}$. Schemes supporting point functions are called “Distributed Point Functions” (DPF).

For a function family $\mathcal{F}$, we define a p -party FSS scheme using 3 algorithms:

- Gen : $\mathbb{N} \times P_{\mathcal{F}} \rightarrow \mathcal{K}^p$ takes as input a security parameter $1^\lambda \in \mathbb{N}$ and a function description $\hat{f} \in P_{\mathcal{F}}$, and outputs p keys $k_1, \dots, k_p$.
- Eval : $\mathcal{K} \times \mathcal{X} \rightarrow \mathbb{G}$ takes as input k_i and a point $x \in \mathcal{X}$, outputs a share of $f(x)$ that we denote as $\llbracket f(x) \rrbracket_i$.
- Decode : $\mathbb{G}^p \rightarrow \mathcal{Y}$ takes as input the shares $\{\llbracket f(x) \rrbracket_1, \dots, \llbracket f(x) \rrbracket_p\}$ and outputs the secret $f(x)$.

Definition 1 (Correctness [2]). A scheme (Gen, Eval, Decode) is correct if, for any function $f \in \mathcal{F}$ and point $x \in \mathcal{X}$, we have:

$$\Pr[\text{Decode}(\text{Eval}(k_1, x), \dots, \text{Eval}(k_p, x)) = f(x)] = 1$$

with $k_1, \dots, k_p \leftarrow \text{Gen}(1^\lambda, \hat{f})$

Definition 2 (Privacy [1]). Let $\text{Leak} : \{0, 1\}^* \rightarrow \{0, 1\}^*$ be a function specifying the allowable leakage. A scheme (Gen, Eval, Decode) is private if, for every set of m corrupted parties $S \subseteq \{1 \dots p\}$, there exists a PPT algorithm Sim (simulator), s.t. for any sequence of function descriptions $(\hat{f}_1, \hat{f}_2, \dots)$ of size polynomial in λ , the outputs of Real and Ideal are computationally indistinguishable:

- Real(1^λ) : $(k_1, \dots, k_p) \leftarrow \text{Gen}(1^\lambda, \hat{f}_\lambda)$; Output $(k_i)_{i \in S}$
- Ideal(1^λ) : Output Sim($1^\lambda, \text{Leak}(\hat{f}_\lambda)$)

DPF by [2] To build their multi-party DPF scheme under dishonest majority ($m < p$), Boyle et al. [2] represented the domain $\{1, \dots, N\}$ as a $\nu \times \nu$ grid (with $\nu = \lceil \sqrt{N} \rceil$). This grid is full of zeros except on the cell (γ_*, δ_*) , with $\alpha = \gamma_* \nu + \delta_*$.

For each row $\gamma \in \{1, \dots, \nu\}$, the Gen algorithm samples q^{p-1} random λ -bit random seeds $s_{\gamma,1} \dots s_{\gamma,q^{p-1}}$. For each seed $s_{\gamma,j}$, the algorithm generates additive shares of a coefficient $a_{\gamma,j}$: $\llbracket a_{\gamma,j} \rrbracket_1, \dots, \llbracket a_{\gamma,j} \rrbracket_p$ (i.e., one share per DPF key). The coefficient is defined as follows $a_{\gamma,j} = 1$ if $\gamma = \gamma_*$, 0 otherwise. Finally, it sets a “correction word” $W \in (\mathbb{F}_q)^\nu$ such that $W + \sum_{i=1}^{q^{p-1}} G(s_{\gamma_*,j}) = e_{\delta_*} \cdot \beta$ (with e_δ a unit vector equal to 1 on index δ , 0 otherwise). Each key k_i contains the correction word, their share of the coefficients $\llbracket a_{\gamma,j} \rrbracket_i$ (for all rows γ and all $j \in \{1, \dots, q^{p-1}\}$), and it contains all the seeds $s_{\gamma,j}$ for which $\llbracket a_{\gamma,j} \rrbracket_i \neq 0$. This last condition (on the seed inclusion in a key) ensures that there is at least one seed unknown to an adversary composed of $p-1$ out of p parties.

The Eval algorithm represents the input x as a tuple (γ, δ) , expands the corresponding seeds $s_{\gamma,j}$, multiplies the expanded seeds with the corresponding shared coefficient $\llbracket a_{\gamma,j} \rrbracket_i$, sums everything with the correction word W , and the share $\llbracket f(x) \rrbracket_i$ is on the δ -th index of the sum vector:

$$\llbracket f(x) \rrbracket_i = v[\delta] \text{ with } v = W + \sum_j \llbracket a_{\gamma,j} \rrbracket_i \cdot G(s_{\gamma,j})$$

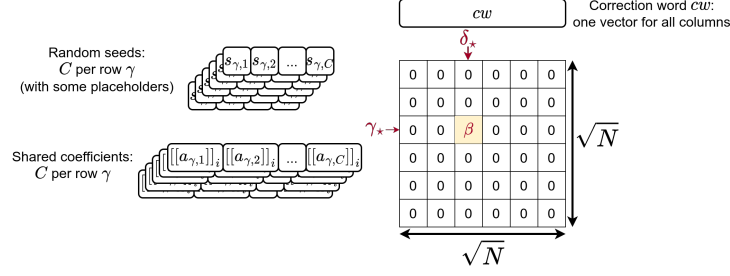


Fig. 1: Structure of our DPF keys

The Decode algorithm is a basic additive share decoding: $\sum_i \llbracket f(x) \rrbracket_i = f(x)$.

As we optimize [2], Algorithm 1 (describing our scheme) follows roughly the same structure as their scheme. The *only difference* is the matrix sampling in Lines 8 and 9. We then refer to Algorithm 1 for more details.

An element could surprise the reader: the number of random seeds $\nu \times q^{p-1}$. Such a large number is necessary, so an adversary cannot infer information about the secret function based on the distribution of the shares $\llbracket a_{\gamma,j} \rrbracket_i$. These shares can be structured as matrix shares A_γ with $A_\gamma[i, j] = \llbracket a_{\gamma,j} \rrbracket_i$. To preserve function privacy, each matrix must contain all combination of additive shares summing to 1 (if $\gamma = \gamma_*$) or 0 (if $\gamma \neq \gamma_*$). If the matrices do not contain all possible combinations, an adversary (owning up to $p - 1$ keys out of p) can recover γ_* based on the share distribution [2]. Since there exists q^{p-1} combinations of p shares of 0 (resp. of 1) in $\mathbb{F}_q$, the key generator must sample q^{p-1} random seeds.

3 Our honest-majority scheme

The main scalability bottleneck in [2] lies in the size of the matrices of shares, we improve their scheme by eliminating its dependence on the field size; thanks to the honest-majority assumption ($m < p/2$). While Boyle et al. [2] had assumed a dishonest majority ($m < p$), the honest-majority assumption enables us to redesign the matrices, resulting in more compact keys. Algorithm 1 presents our multi-party DPF scheme, and Figure 1 an overview of our DPF keys.

Our matrix sampling (described in the function `MatrixOfShares` of Algorithm 1) generates a matrix A that distributes shares among all possible combinations of $m + 1$ parties out of p . For each combination S_j of $m + 1$ parties, the function samples $m + 1$ shares of the secret coefficient a (using additive secret sharing) and assigns each share to the cell of A ; $A[i, j] = \llbracket a \rrbracket_i$ if $i \in S_j$, 0 otherwise. The secret coefficient a is either set to 0 or 1 depending on whether the value x being evaluated matches α (the non-zero point).

While in [2], each matrix of shares has p rows and q^{p-1} columns, our construction produces matrices with p rows and $\binom{p}{m+1}$ columns. As in [2], we sample one random seed per column. The i -th key contains the i -th row of the matrix and includes the j -th seed if the corresponding cell $A[i, j]$ is not null.

Algorithm 1 Honest-majority DPF scheme adapted from [2].

```

1: function MATRIXOFSHARES( $a$ )
2:   Initialize  $A$  an  $p \times C$  matrix with zeros and the counter  $k$  to 1.
3:   for each set of parties  $S_j$  in the set of all combinations of  $m + 1$  of  $p$  do
4:     Sample  $m + 1$  shares of the value  $a$ :  $\{\llbracket a \rrbracket_{1,i} \dots \llbracket a \rrbracket_{m+1,i}\}$ .
5:     for each  $i$  in  $S_j$  do  $A[i, j] \leftarrow \llbracket a \rrbracket_{k,i}$  and increment  $k$ .
   return  $A$ 

6: function Gen( $\alpha, \beta, p, m, 1^\lambda$ )
7:   Represent  $\alpha$  as a pair  $\alpha = (\gamma_*, \delta_*)$  with  $\gamma_*, \delta_* \in \{0 \dots \nu\}$ .
8:   Sample  $A_1, \dots, A_\nu$  s.t. for all  $\gamma \neq \gamma_*$ ,  $A_\gamma \leftarrow \text{MatrixOfShares}(0)$ .
9:   Sample  $A_{\gamma_*} \leftarrow \text{MatrixOfShares}(1)$ .
10:  Choose randomly and independently  $\nu \cdot C$  seeds  $s_{1,1}, \dots, s_{\nu,C} \in \{0, 1\}^\lambda$ .
11:  Set the correction words  $W \in \mathbb{G}^\nu$  s.t.  $W + \sum_{j=1}^C G(s_{\gamma_*,j}) = e_{\delta_*} \cdot \beta$ .
12:  for  $i \in \{1 \dots p\}, j \in \{1 \dots C\}, \gamma \in \{1 \dots \nu\}$  do
13:    if  $A_\gamma[i, j] \neq 0$  then  $\sigma_{i,\gamma,j} \leftarrow (s_{\gamma,j}, A_\gamma[i, j])$ .
14:    else  $\sigma_{i,\gamma,j} \leftarrow (0, 0)$ . ▷ Receives no seed and no coefficient share
15:  Set  $\sigma_{i,\gamma} \leftarrow (\sigma_{i,\gamma,1} || \dots || \sigma_{i,\gamma,C})$  for all  $1 \leq i \leq p, 1 \leq \gamma \leq \nu$ .
16:  return  $(k_1, \dots, k_p)$  with  $k_i = (\sigma_{i,1} || \dots || \sigma_{i,\nu} || W)$  for all  $1 \leq i \leq p$ .

17: function Eval( $k_i, x$ )
18:  Represent  $x$  as a pair  $x = (\gamma, \delta)$  with  $\gamma, \delta \in \{0 \dots \nu\}$ .
19:  Parse  $k_i = ((s_{1,1}, A_1[i, 1]) || \dots || (s_{1,C}, A_1[i, C]) || \dots || (s_{\nu,C}, A_\nu[i, C]) || W)$ .
20:  return  $y_i[\delta]$  with  $y_i \leftarrow A_\gamma[i, 1] \cdot W + \sum_{j=1}^C A_\gamma[i, j] \cdot G(s_{\gamma,j})$ .

21: function Decode( $\llbracket y \rrbracket_1, \dots, \llbracket y \rrbracket_p$ ) return  $\sum_{i=1}^p \llbracket y \rrbracket_i$ .
```

As in [2], it is necessary that (for any given γ) at least one seed $s_{\gamma,j}$ remains unknown to the adversary. With all the seeds, they could unmask the correction word W and recover β . Our scheme provides each seed to $m + 1$ parties, so *under honest majority*, at least one combination of $m + 1$ out of p parties contains only honest agents. Thus, there is at least one seed unknown to an adversary.

Our optimization cannot be extended to dishonest majority. Indeed, with $m > p/2$, the adversary would know all the seeds because there would be at least one corrupted party in each combination of $m + 1$ out of p parties.

Based on these intuitions, we can consider the following security theorem:

Theorem 1. *Let $\lambda \in \mathbb{N}$, $N, p \in \mathbb{N}$, then the tuple $(\text{Gen}, \text{Eval}, \text{Decode})$ as described in Algorithm 1 is an FSS scheme for the family of all point functions with $\alpha \in \{1, \dots, N\}$ and any $\beta \in \mathbb{F}_q$.*

Assuming that there exists a secure PRG, then this scheme is correct and private against at most m semi-honest parties with $m < p/2$.

Proof. As we modified only slightly the scheme of [2] (i.e., redesigned the matrix of shares based on the honest-majority assumption), our proof follows the same structure as theirs. We provide a proof sketch, and refer to [2] for more details.

The correctness can be verified by an easy arithmetic exercise considering successively three cases: (1) $\gamma \neq \gamma_*$, (2) $\gamma = \gamma_*$ and $\delta \neq \delta_*$, and (3) $\gamma = \gamma_*$ and $\delta = \delta_*$. Using the $\sqrt{N} \times \sqrt{N}$ grid (illustrated in Figure 1), we represent any input x as (γ, δ) and α as (γ_*, δ_*) .

For privacy, we must show that there exists a simulator that outputs samples from a distribution that is computationally indistinguishable from the distribution of the real DPF keys. We propose to study separately: the random seeds $s_{\gamma,j}$, the correction word W , and the coefficient shares $A_\gamma[i, j]$.

The simulation is straightforward: for each (simulated) seed, sample a random seed $\widetilde{s}_{\gamma,j}$; for the correction word, sample a random vector $\widetilde{W}$; for the coefficients, for each S_j combination of $m+1$ parties of p , for each i in S_j , sample a random value and store it in $\widetilde{A}_\gamma[i, j]$ (the rest of $\widetilde{A}_\gamma$ is null). The simulator can return “simulated” keys based on these elements.

Since both the real and simulated seeds are randomly sampled, the simulator’s output distribution ($\widetilde{s}_{\gamma,j}$) is computationally indistinguishable from that induced by the distribution of a single output of Gen.

The correction word W is a secret vector (i.e., $e_{\delta_*} \cdot \beta$) masked with the output of $\binom{p}{m+1}$ seeded PRGs. Remember that a key k_i contains a seed $s_{\gamma,j}$ only if the i is part of the j -th combination of m out of p parties. So, under honest majority, there is always at least one seed unknown to the adversary controlling m out of p parties. Hence, the correction word W is computationally indistinguishable from the randomly sampled $\widetilde{W}$, because it is masked with the output of (at least) one PRG seeded with a seed unknown to the adversary [6].

Finally, each coefficient is shared between $m+1$ parties, so an adversary controlling m parties cannot distinguish real shares from random values $\widetilde{A}_\gamma[i, j]$ provided by the simulator. $\square$

Key size optimization: Instead of using a $\sqrt{N} \times \sqrt{N}$ grid, we should use a grid with $\sqrt{N}(C)^{-1}$ rows and $\sqrt{N} \cdot C$ columns, with $C = \binom{p}{m+1}$. This “non-square” grid leverages the fact that each row requires C seeds, and a unique vector W for all columns. Thanks to this trick, we obtain a better key size: $O(\sqrt{N} \cdot C(\lambda + \log q))$.

Extension to comparison functions Boyle et al. [2] presented a simple adaptation of their DPF to support comparison functions; functions such that $f(x) = \beta$ when $x \leq \alpha$, 0 otherwise. These schemes have applications notably in machine learning [4,8]. We can naively reuse our optimization on this other scheme.

4 Key size comparison

This section compares our optimized scheme to existing schemes in order to identify asymptotic and practical key size reductions.

We focus our comparison on schemes supporting all possible DPF applications; excluding schemes based on elliptic curves that support a limited number of applications due to their non-linear decoding [8] (e.g., do not support PIR).

Asymptotic Our scheme provides a key size of $O(\sqrt{N \cdot \binom{p}{m+1}} \log q)$, which is clearly better than the key size of [2] (i.e., $O(\sqrt{N} q^{\frac{p-1}{2}} \log q)$).

As Bunn et al. [5] built an honest-majority scheme with $O(\sqrt[4]{N})$ key size upon [2], we can comment how we distinguish from them. Their paper does not modify [2], but combine it with replicated secret sharing to reduce the dependency on N (i.e., $O(\sqrt[4]{N})$ instead of $O(\sqrt{N})$). However, their approach worsened the exponential factors already in [2]; as shown in our benchmark below. On the contrary, we leverage the honest-majority scheme to avoid exponential factors present in [2], but we maintained the same dependence on N .

The IT DPF by [5] has a key size comparable to ours: $O(\sqrt{N} \cdot \binom{p}{m+1} \log q)$. However, our approach saves a factor $\sqrt{\binom{p}{m+1}}$ compared to them, yielding significant key size reductions in practice.

While these asymptotic comparisons are informative, they are often insufficient to assess practical performance. Bunn et al. [5] exemplify this problem: although they substantially reduced the dependence on the domain size N , they kept exponential factors without providing any concrete efficiency analysis. Therefore, we present a detailed comparison based on exact key sizes to offer a more accurate assessment.

Exact As we aimed to avoid the exponential factor q^p (for outputs in $\mathbb{F}_q$), we start by studying the dependency on q . Figure 2 compares key sizes for varying prime moduli. Our benchmark includes a curve “Trivial scheme” corresponding to the most trivial DPF: sharing the function truth table (i.e., $O(N)$ key size). This curve serves as baseline to identify impractical solutions. For any $q > 5$, the PRG-based solutions [2,5] have keys orders of magnitude larger than those of this trivial solution, while the IT DPF of [5] and ours are below.

Recently, Boyle [1] showed that, for a composite modulus $m = q_1 q_2 \dots q_l$, we can use the Chinese Remainder Theorem (CRT) to replace q^p with $\sum q_i^p$. Figure 2a compares key sizes for arbitrary moduli, but the variations make the figure poorly readable. Instead, Figure 2c compares key sizes for primorial moduli; a primorial is the product of the first n primes. Primorials are the best case of this CRT trick as they provide composite moduli with the smallest primes possible.

Even with the CRT trick, the existing PRG-based schemes [2,5] provides key sizes larger than the trivial scheme for any modulus above 210. As their key sizes are impractical, we exclude these schemes from our other figures to focus on the comparison of our scheme to practical schemes.

Figure 2c also shows that the key size of the IT scheme of [5] grows faster with the modulus than ours. This phenomenon is explained by the fact that our key size is dominated by components conditioned by a security parameter that is independent of the modulus.

Figure 3a compares the practical schemes for varying function domain sizes. **Our key size is 2.4× smaller than the best existing scheme.** Moreover, Figure 3b shows that **our scheme has a better scaling with the number of parties** thanks to the factor $\sqrt{\binom{p}{m+1}}$ identified in the asymptotic comparison.

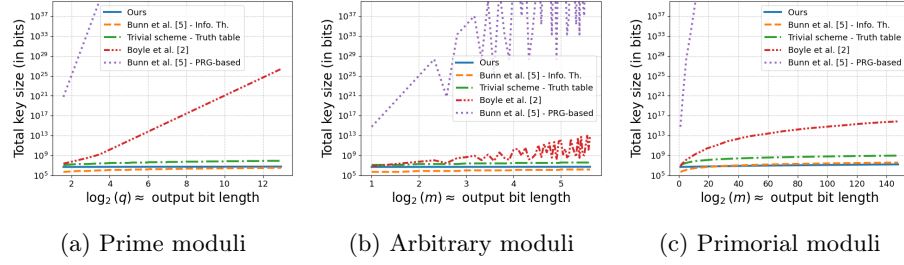
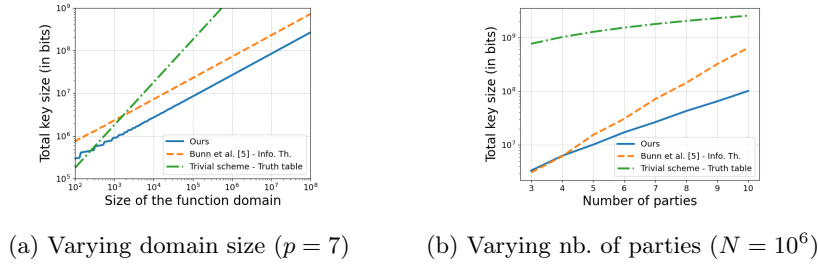
Fig. 2: Key size of various DPF schemes for varying moduli ($p = 7$).

Fig. 3: Key size of the most efficient DPF schemes.

Conclusion Our optimization based on the honest-majority assumption transformed a PRG-based DPF [2] with impractical key sizes into the DPF with the smallest key sizes. Our work proves that, like in two- and three-party schemes, PRG is a promising primitive to build multi-party DPF with compact keys.

Acknowledgments. This work was supported by the Netherlands Organization for Scientific Research (De Nederlandse Organisatie voor Wetenschappelijk Onderzoek) under NWO:SHARE project [CS.011].

References

1. Boyle, E.: Function Secret Sharing and Homomorphic Secret Sharing (2022)
2. Boyle, E., Gilboa, N., Ishai, Y.: Function Secret Sharing. In: Advances in Cryptology - EUROCRYPT 2015 (2015). https://doi.org/10.1007/978-3-662-46803-6_12
3. Boyle, E., Gilboa, N., Ishai, Y.: Function Secret Sharing: Improvements and Extensions. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (Oct 2016). <https://doi.org/10.1145/2976749.2978429>
4. Boyle, E., Gilboa, N., Ishai, Y.: Secure Computation with Preprocessing via Function Secret Sharing. In: Theory of Cryptography (2019). https://doi.org/10.1007/978-3-030-36030-6_14
5. Bunn, P., Kushilevitz, E., Ostrovsky, R.: CNF-FSS and Its Applications. In: Public-Key Cryptography 2022 (2022). https://doi.org/10.1007/978-3-030-97121-2_11

6. Corrigan-Gibbs, H., Boneh, D., Mazières, D.: Riposte: An Anonymous Messaging System Handling Millions of Users. In: 2015 IEEE Symposium on Security and Privacy (May 2015). <https://doi.org/10.1109/SP.2015.27>
7. Gilboa, N., Ishai, Y.: Distributed Point Functions and Their Applications. In: EUROCRYPT 2014 (2014). https://doi.org/10.1007/978-3-642-55220-5_35
8. Kumar, C., Patranabis, S., Mukhopadhyay, D.: Compact Key Function Secret Sharing with Non-linear Decoder. IACR Communications in Cryptology **1**(2) (Jul 2024). <https://doi.org/10.62056/a3c3c3w9p>
9. Zyskind, G., Yanai, A., Pentland, A.S.: High-Throughput Three-Party DPFs with Applications to ORAM and Digital Currencies. In: Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (2024)